

Angular Moderno

11 - Signals na Prática

Aula 11 — Signals na Prática: Carrinho de Compras

Nesta aula, transformaremos nosso projeto de e-commerce em algo mais próximo de um sistema real. Vamos implementar um carrinho de compras completo utilizando **Signals do Angular 21+** (sem NgModules, sem RxJS, com código limpo e reativo do início ao fim).



Tema da Aula

Signals na Prática: Carrinho de Compras

Chegou a hora de colocar os Signals para trabalhar de verdade. Vamos construir um carrinho de compras funcional, reativo e integrado ao catálogo de produtos. Tudo dentro do nosso projeto de e-commerce progressivo.

O que já temos

Catálogo de produtos, @for, @if, comunicação entre componentes e estado reativo com signal()

O que vamos construir

Um carrinho de compras com estado separado, valores derivados e atualização automática da UI



Objetivos da Aula

- Novo estado reativo para o carrinho
Criar um `signal()` separado para representar os itens do carrinho, independente do catálogo de produtos.
- Manipular estado com `update()`
Aprender a atualizar o Signal de forma imutável e segura usando o método `update()`.
- Valores derivados com `computed()`
Calcular automaticamente quantidade e total do carrinho sem lógica extra no template.
- Fluxo real de e-commerce
Demonstrar um ciclo completo:
catálogo → adicionar produto → exibir carrinho → calcular total.

Conceito Inicial: Dois Estados Diferentes

Antes de escrever qualquer código, é fundamental entender **por que precisamos de um novo Signal**. Nossa aplicação terá dois estados distintos e com responsabilidades separadas.

Catálogo de Produtos

O que representa: A lista completa de produtos disponíveis para compra.

Origem: Normalmente vinda de uma API ou arquivo de dados.

Comportamento: Geralmente estático ou raramente alterado durante a sessão.

Carrinho de Compras

O que representa: Os itens que o usuário escolheu comprar.

Origem: Gerado pelas ações do usuário (cliques em "Adicionar").

Comportamento: Muda constantemente conforme o usuário interage com a loja.

❗ Separar responsabilidades desde o início é a base de um código escalável. Cada estado deve ter sua própria fonte da verdade.

Implementação Inicial vs. Arquitetura Profissional

O que faremos NESTA aula

O carrinho será implementado **diretamente dentro do componente**. O estado, os métodos e os valores derivados ficam todos em `lista-produtos.ts`.

Isso é **intencional e didático**. Nosso foco aqui é aprender Signals em profundidade! Não é, ainda, arquitetura.

O que faremos em aulas futuras

Esse código será **refatorado para um Service**. O carrinho passará a ser um estado global, compartilhado entre diferentes componentes e rotas da aplicação.

Esse fluxo (*primeiro funcional, depois arquitetura*) é exatamente como acontece no mundo real.

⚠ Não se preocupe, por enquanto, se o código parecer "fora do lugar". **Isso é proposital**. Aprendemos primeiro a mecânica, depois a organização.

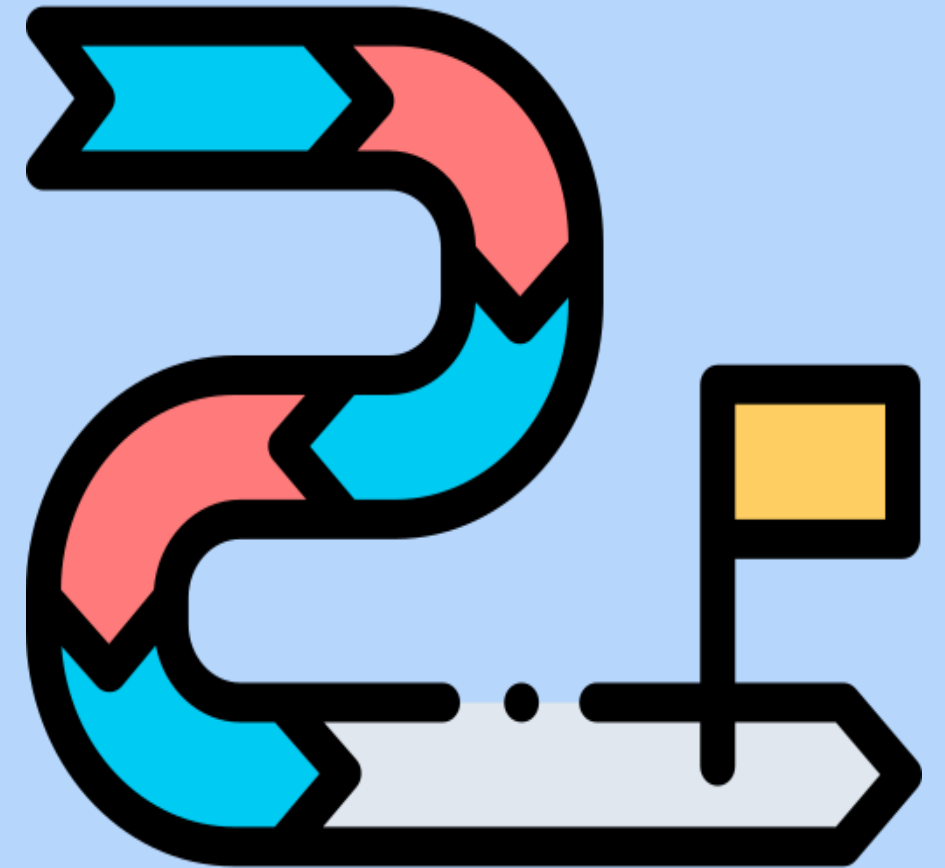
Parte Prática Passo a Passo

11 Passos até o Carrinho Funcional

A partir de agora, cada passo será detalhado em um slide separado.

Siga a ordem rigorosamente!

Cada etapa depende da anterior. Não pule nenhuma etapa.



Parar o Servidor

Por que parar o servidor?

Antes de modificar os arquivos do projeto, é uma boa prática parar o servidor de desenvolvimento. Isso evita comportamentos inesperados durante edições, especialmente quando alteramos estruturas de componentes ou adicionamos novos membros à classe.

Com o servidor parado, você pode fazer todas as alterações com segurança e só reiniciá-lo quando o código estiver pronto.

Como parar

No terminal onde o `ng serve` está rodando, pressione:

```
CTRL + C
```

O terminal irá confirmar que o processo foi encerrado. Aguarde antes de continuar.

✔ Servidor parado com sucesso. Agora podemos editar os arquivos.

Criar o Estado do Carrinho

Abra o arquivo `lista-produtos.ts` e adicione o novo Signal dentro da classe do componente, logo abaixo dos signals existentes.

```
carrinho = signal<{ nome: string; preco: number }[]>([]);
```

Tipo explícito

Usamos `signal<{ nome: string; preco: number }[]>` para que o TypeScript saiba exatamente qual estrutura de dado esperar no array.

Valor inicial vazio

O carrinho começa como um array vazio `[]`. Faz sentido: nenhum produto foi adicionado ainda quando a página carrega.

Estado separado

Este é um Signal **completamente independente** do catálogo de produtos. Dois estados distintos, duas fontes da verdade.

Criar o Método `adicionarAoCarrinho()`

Ainda em `lista-produtos.ts`, adicione o método abaixo na classe do componente. Este método será chamado quando o usuário clicar em "Adicionar ao Carrinho".

```
adicionarAoCarrinho(produto: { nome: string; preco: number }) {  
  this.carrinho.update(listaAtual => [  
    ...listaAtual,  
    produto  
  ]);  
}
```

`update()`

Recebe a lista atual como argumento e retorna a nova lista. O Signal atualiza automaticamente qualquer parte da UI que o lê.

Spread operator (...)

O `...listaAtual` copia todos os itens existentes. Nunca modificamos o array original — criamos um novo. Isso é **imutabilidade**.

Reatividade automática

Ao chamar `update()`, o Angular detecta a mudança e re-renderiza automaticamente todo template que depende de `carrinho()`.

Adicionar Botão no Template `lista-produtos.html`

No template HTML do componente, **dentro do bloco** `@for`, insira um botão que adiciona produto ao carrinho:

```
@for (item of produtos(); track $index) {  
  <app-produto  
    [nome]="item.nome"  
    [preco]="item.preco"  
    (produtoSelecionado)="exibirProduto($event)"  
  ></app-produto>  
  
  <button (click)="adicionarAoCarrinho(item)">  
    Adicionar ao Carrinho  
  </button>  
}
```

Passo 5 de 11

Exibir os Itens do Carrinho no Template **lista-produtos.html**

Na parte final do template:

```
<hr />

<h2>Carrinho:</h2>

@for (item of carrinho(); track $index) {
  <p>{{ item.nome }} - R$ {{ item.preco.toFixed(2) }}</p>
}
```

Criar `quantidadeCarrinho` com `computed()`

Em `lista-produtos.ts`, adicione um `computed` que calcula automaticamente o número de itens no carrinho.

```
quantidadeCarrinho = computed(() => this.carrinho().length);
```

O que é um `computed()`?

Um `computed()` é um Signal derivado — ele lê outros Signals e recalcula seu valor automaticamente sempre que alguma dependência muda. Você nunca precisa chamar manualmente.

É como uma fórmula em uma planilha: mude um valor, os resultados atualizam sozinhos.

Por que não usar `carrinho().length` diretamente no template?

Você poderia. Mas ao encapsular em um `computed()`, você:

- Nomeia a intenção do valor
- Centraliza a lógica no componente
- Facilita testes e manutenção futura

Criar `totalCarrinho` com `computed()`

Adicione mais um computed para calcular o valor total dos itens no carrinho. Este é um exemplo perfeito de lógica que **não deve ficar no template**.

```
totalCarrinho = computed(() => {  
  return this.carrinho().reduce((total, item) => total + item.preco, 0);  
});
```

Array.reduce()

O método `reduce()` percorre cada item do array e acumula um resultado. Aqui, somamos o `preco` de cada produto até chegar no total final. O valor inicial do acumulador é `0`.

Reatividade em cadeia

Como `totalCarrinho` lê `this.carrinho()`, ele é automaticamente recalculado cada vez que um produto é adicionado. Nenhum código extra é necessário. Os Signals cuidam disso.

Computed é somente leitura

Você não pode fazer `totalCarrinho.set(0)`. Computed Signals são derivados. Seu valor sempre reflete o estado dos Signals dos quais dependem.

Exibir os Totais no Template

No template HTML, abaixo da lista do carrinho, exiba os valores calculados pelos computed Signals.

```
<p><strong>Itens no carrinho:</strong> {{ quantidadeCarrinho() }}</p>
<p><strong>Total do Carrinho:</strong> R$ {{ totalCarrinho().toFixed(2) }}</p>
```

Lendo Computed Signals

Assim como Writable Signals, os Computed Signals também precisam de `()` para serem lidos no template. A sintaxe é idêntica: `quantidadeCarrinho()` e `totalCarrinho()`.

Formatação monetária com `toFixed(2)`

O método `.toFixed(2)` formata o número com exatamente 2 casas decimais — essencial para valores monetários. Garante que `9.9` apareça como `9.90` para o usuário.

- ✔ Neste ponto, já temos um carrinho funcional com reatividade completa!
Os próximos passos organizam a interface.

Organizar o Layout. Deixe `lista-produtos.html` assim:

Com o carrinho funcionando, vamos organizar o template separando a seção de produtos da seção do carrinho.

```
<!-- Seção de Produtos -->
<section>
  <h2>Produtos:</h2>

  @for (item of produtos(); track item.nome) {
    <app-produto
      [nome]="item.nome"
      [preco]="item.preco"
      (produtoSelecionado)="exibirProduto($event)"
    ></app-produto>

    <button (click)="adicionarAoCarrinho(item)">Adicionar ao
Carrinho</button>
  }

  <h2>
    Produto selecionado:
    @if (produtoSelecionado()) {
      <strong style="color: green">{{ produtoSelecionado() }}</strong>
    } @else {
      <span style="color: gray">Nenhum</span>
    }
  </h2>
```

```
<p>
  <button (click)="adicionarProduto()">
    Adicionar Produto
  </button>
</p>
<p>
  <button (click)="substituirProdutos()">
    Substituir Lista
  </button>
</p>
<h2>Total de produtos: {{ totalProdutos() }}</h2>
<h3>Valor total: R$ {{ valorTotal() }}</h3>
</section>
<!-- Seção do Carrinho -->
<section>
  <hr />
  <h2>Carrinho:</h2>
  @for (item of carrinho(); track $index) {
    <p>{{ item.nome }} - R$ {{ item.preco.toFixed(2) }}</p>
  }
  <p><strong>Itens no carrinho:</strong> {{ quantidadeCarrinho() }}</p>
  <p><strong>Total do Carrinho:</strong> R$ {{ totalCarrinho().toFixed(2) }}</p>
</section>
```



Por que Organizar o Layout?



Separação visual clara

Usar `<section>` e `<hr />` separa logicamente as duas áreas da tela: o catálogo de produtos e o carrinho. O usuário entende imediatamente o que é cada parte.



Código mais legível

Comentários como `<!-- Seção de Produtos -->` tornam o template muito mais fácil de manter. Quem lê o Código, incluindo você mesmo no future, entende a estrutura em segundos.



Clareza para o usuário

Um carrinho de compras real sempre separa visualmente o catálogo do resumo da compra. Começar com essa separação agora prepara a base para o design final.

Passo 10 de 11

Reiniciar o Servidor

Comando para reiniciar

Com todas as alterações salvas nos arquivos `.ts` e no template, volte ao terminal e execute:

```
ng serve
```

Aguarde a compilação. O Angular irá verificar os tipos e compilar o projeto. Se houver algum erro de TypeScript, ele aparecerá aqui. Este é o momento certo para corrigir antes de testar.

Testar a Aplicação

Com o servidor rodando, é hora de validar que tudo funciona corretamente. Siga este roteiro de testes no navegador:

01

Adicionar o primeiro produto

Clique em "Adicionar ao Carrinho" em qualquer produto. O item deve aparecer imediatamente na seção do Carrinho, abaixo do `<hr />`.

03

Adicionar mais produtos

Clique em outros produtos. Veja a lista do carrinho crescer, a quantidade aumentar e o total ser recalculado automaticamente a cada clique.

02

Verificar quantidade e total

Confirme que "Itens no carrinho" mostra **1** e que o "Total do Carrinho" exibe o preço correto com 2 casas decimais.

04

Adicionar o mesmo produto duas vezes

O mesmo item deve aparecer duas vezes no carrinho (comportamento esperado nesta fase. Remoção e agrupamento serão tratados futuramente).

Consolidação: O que Construímos

Vamos revisar os três pilares de Signals que utilizamos nesta aula e como eles se conectam para criar um carrinho totalmente reativo.



Writable Signal

```
carrinho = signal([])
```

A fonte da verdade do carrinho.

Modificado via `update()` de forma imutável. Cada mudança dispara a reatividade.



Computed Signals

```
quantidadeCarrinho e totalCarrinho
```

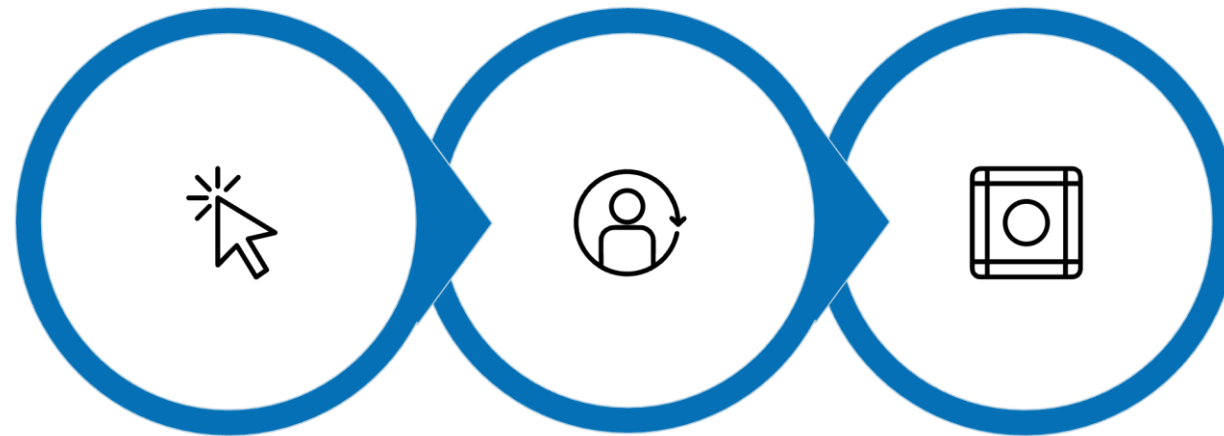
Derivam seus valores do carrinho automaticamente. Recalculam sozinhos. Somente leitura.



Template Reativo

```
carrinho(), quantidadeCarrinho(),  
totalCarrinho()
```

O Angular re-renderiza automaticamente apenas o que mudou. Zero código extra para atualizar a UI.



Clique

update()

Template



Erros Comuns. Fique Atento!

Esquecer os parênteses `()`

Errado: `{{ carrinho.length }}` **Certo:** `{{ carrinho().length }}` Signals são funções. Sem `()`, você acessa o objeto Signal, não seu valor. O Angular não vai reclamar — mas exibirá dados errados.

Usar `push()` diretamente

Errado: `this.carrinho().push(produto)` **Certo:** `this.carrinho.update(lista => [...lista, produto])` O `push()` muta o array original sem notificar o Signal. A UI não atualiza.

Misturar variável comum com Signal

Errado: `let total = 0;` atualizado manualmente.

Certo: `totalCarrinho = computed(...)` Variáveis comuns não são reativas. O template não sabe que elas mudaram.

Calcular totais diretamente no template

Evite: Lógica como `carrinho().reduce(...)` inline no HTML.

Prefira: Encapsular em `computed()` no componente. O template deve ser declarativo e limpo.

Ajustes Finais e Organização do Projeto

Com o carrinho funcionando, dedique alguns minutos para organizar o código. Um componente bem organizado é tão importante quanto um componente funcional.

Boas práticas de código

- Use `.toFixed(2)` em **todo** valor monetário exibido ao usuário
- Remova textos de placeholder que vieram do template inicial (`<p>lista-produtos works!</p>`)
- Use comentários para separar blocos lógicos no template e na classe
- Prefira nomes descritivos: `adicionarAoCarrinho` é melhor que `addItem`

Organização dos Signals na classe

- Agrupe os Signals e Computed de forma lógica:

 Código organizado hoje = manutenção fácil amanhã. Especialmente importante antes da refatoração para Services.

Código Final Completo: `lista-produtos.ts`

```
import { Component, signal, computed, effect } from '@angular/core';
import { Produto } from '../produto/produto';

@Component({
  selector: 'app-lista-produtos',
  imports: [Produto],
  templateUrl: './lista-produtos.html',
  styleUrls: ['./lista-produtos.css'],
})

export class ListaProdutos {

  // SIGNALS

  produtos = signal([
    { nome: 'Notebook', preco: 3800 },
    { nome: 'Mouse', preco: 179 }
  ]);

  produtoSelecionado = signal<string | null>(null);

  carrinho = signal<{ nome: string; preco: number }[]>([]);

  // COMPUTED

  totalProdutos = computed(() => this.produtos().length);

  valorTotal = computed(() => {
    return this.produtos()
      .reduce((total, item) => total + item.preco, 0);
  });

  quantidadeCarrinho = computed(() => this.carrinho().length);

  totalCarrinho = computed(() => {
    return this.carrinho().reduce((total, item) => total + item.preco, 0);
  });
}
```

```
constructor() {
  effect(() => {
    console.log('Lista de produtos alterada:', this.produtos());
  });

  effect(() => {
    console.log('Valor total atualizado:', this.valorTotal());
  });

  effect(() => {
    if (typeof document !== 'undefined') {
      document.title = `${this.totalProdutos()} Minha Loja`;
    }
  });
}

exibirProduto(nome: string) {
  //console.log('Produto selecionado:', nome);
  this.produtoSelecionado.set(nome);
}

// update() modifica baseado no estado atual
adicionarProduto() {
  this.produtos.update(listaAtual => [
    ...listaAtual,
    { nome: 'Teclado', preco: 250 }
  ]);
}

// set() substitui completamente
substituirProdutos() {
  this.produtos.set([
    { nome: 'Produto novo', preco: 999 }
  ]);
}

adicionarAoCarrinho(produto: { nome: string; preco: number }) {
  this.carrinho.update(listaAtual => [
    ...listaAtual,
    produto
  ]);
}
}
```

Resultado da Aula

3

Signals criados

produtos, carrinho e produtoSelecionado

2

Computeds criados

quantidadeCarrinho e totalCarrinho

11

Passos concluídos

Do zero até o carrinho funcionando

O que temos agora

- Sistema de carrinho completamente funcional
- Atualização automática da UI sem código manual
- Quantidade e total calculados reativamente
- Código organizado, limpo e sem NgModules
- Base sólida para refatoração futura em Services
- Separação e organização entre catálogo de produtos e carrinho

O que vem a seguir

- Angular Router — navegação entre páginas
- A necessidade de estado global ficará evidente
- Refatoração do carrinho para um Service
- Signals globais compartilhados entre componentes
- Persistência de estado com localStorage

Regras Importantes Abordadas Nesta Aula: Resumo

Angular 21+ Standalone

Projeto baseado em Standalone Components (sem uso de `NgModule`).

Sem RxJS nesta aula

Nenhum `Observable`, `Subject` ou pipe do RxJS. Toda reatividade é feita exclusivamente com `signal()` e `computed()`.

Sintaxe moderna

Uso de `@for` e `@if` no lugar de `*ngFor` e `*ngIf`. Template completamente atualizado para Angular 17+.

Código completo e funcional

Nenhum passo foi pulado. Cada trecho de código mostrado é completo e pode ser copiado diretamente para o projeto.

📌 Esta aula faz parte de um curso progressivo. O código evolui a cada aula — do simples ao profissional, passo a passo.



Parabéns! Aula Concluída.

Você acabou de implementar um carrinho de compras funcional com Signals do Angular 21+. Isso é reatividade real, sem magia, sem boilerplate. Apenas código moderno e limpo.



Revise o código

Certifique-se de que seu projeto está organizado conforme o slide de ajustes finais antes de prosseguir.



Faça um commit

Salve o progresso:

```
git add .  
git commit -m "aula-11: carrinho  
com signals"
```



Próxima aula

Angular Router: Adicionando navegação ao nosso e-commerce e descobrindo por que precisamos de estado global.

Dúvidas

?

